

RETAILMART V3 • SQL

SQL JOINS Part 1 — INNER, LEFT, RIGHT



WhatsApp Announcement

SQL JOINS — CONNECTING TABLES

Imagine a wedding where the bride's family sits on the left, groom's on the right. Some guests are related to BOTH families. INNER JOIN finds those common guests. LEFT JOIN keeps ALL of bride's side. Today, SQL learns to CONNECT tables!

Today's Focus:

- INNER JOIN — only matching rows from both tables
- LEFT JOIN — all from left + matches from right
- RIGHT JOIN — mirror of LEFT JOIN
- Multi-table JOINS — connecting 3, 4, 5+ tables

JOINS are the #1 most-asked SQL interview topic. Master this and you crack 80% of SQL interviews!

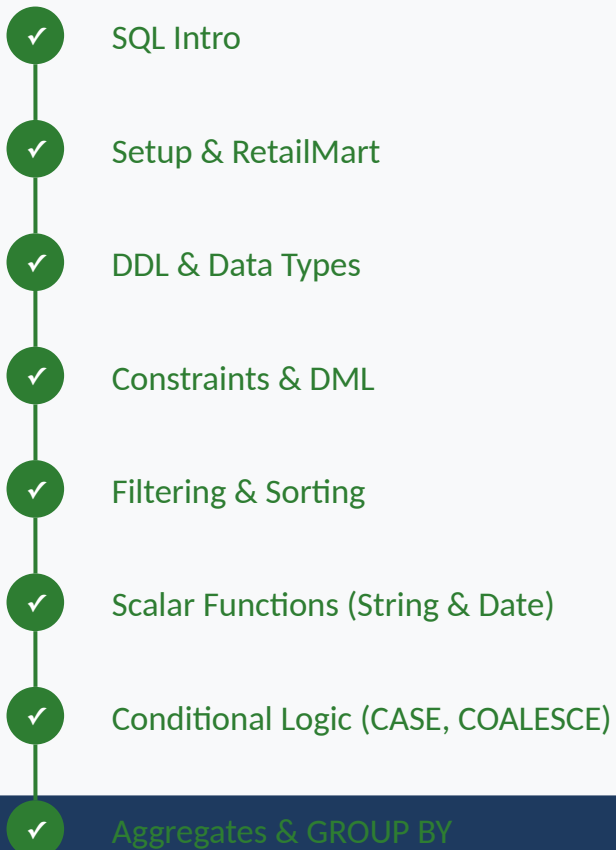
— Siraj Sir

#Day9 #PostgreSQL #JOINS #InnerJoin #LeftJoin

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap — earlier sessions	5 mins	Aggregates, GROUP BY, HAVING from Aggregates & Grouping, Why tables are split (normalization)
Part 1: INNER JOIN	45 mins	Why tables are split — the need for JOINS, INNER JOIN — only matching rows from both tables, Table aliases and column prefixing, 4 practice problems (Easy → Crazy)
Part 2: LEFT JOIN	40 mins	LEFT JOIN — keep ALL from the left table, NULLs from unmatched rows, Anti-join pattern (WHERE IS NULL), 4 practice problems (Easy → Crazy)
Part 3: RIGHT JOIN & Multi-table JOINS	30 mins	RIGHT JOIN — mirror of LEFT JOIN, Chaining 3, 4, 5+ tables, JOIN + WHERE + GROUP BY combo, 4 practice problems (Easy → Crazy)
Quiz + Summary & Wrap-up	20 mins	Quick quiz (5 questions), Common mistakes gallery, Interview bank — JOINS, Syntax Reference Card, Homework + Joins Part 2 preview

YOUR SQL JOURNEY



Memory Check

SELECT, WHERE, LIKE, ORDER BY — finding data in one table.

earlier sessions: Functions and CASE — transforming data in one table.

GROUP BY, HAVING, COUNT/SUM/AVG — summarizing one table.

Every query so far touched ONE table. Today we break that barrier. JOINS let you combine 2, 3, even 7 tables in a single query. This is where SQL becomes truly powerful.

The Reunion Problem

A school reunion is being organized. There are TWO lists: List A has all graduates (with roll numbers). List B has all RSVPs (with roll numbers). The lists do not match perfectly.

Some graduates RSVP'd (in BOTH lists) — INNER JOIN. Some graduates did not RSVP (in List A only) — LEFT JOIN shows them with 'No RSVP'. Some RSVPs are from non-graduates (in List B only) — RIGHT JOIN shows them. The organizer matches roll numbers — that is the ON condition.

From Story to SQL!

Matching roll numbers = ON graduates.roll_no = rsmps.roll_no

Graduates who RSVP'd = INNER JOIN (only matches)

ALL graduates + RSVP status = LEFT JOIN (all left + matching right)

ALL RSVPs + graduate status = RIGHT JOIN (all right + matching left)

Count per year = JOIN + GROUP BY graduation_year

Normalization — Why We Split Data

Technical:

If we stored customer name, email, phone in EVERY order row, a customer with 100 orders would have their name repeated 100 times. Change the name? Update 100 rows. Miss one? Inconsistent data.

Layman's:

Store customer data ONCE in customers.customers. Store orders in sales.orders with just a cust_id reference. JOIN them when you need both. PRIMARY KEY in one table + FOREIGN KEY in another = the JOIN bridge.

RetailMart Table Relationships

```
-- customers.customers (PK: customer_id)
--   > sales.orders (FK: cust_id)
--       > sales.order_items (FK: order_id)
--           > products.products (FK: prod_id)
--       > sales.payments (FK: order_id)
--       > sales.shipments (FK: order_id)

-- stores.stores (PK: store_id)
--   > stores.employees (FK: store_id)
--   > sales.orders (FK: store_id)

-- products.products > core.dim_brand (FK: brand_id)
--   > core.dim_category (FK: category_id)

-- Follow FK to PK – that is the JOIN path!
```

Without Normalization (The Problem)

Imagine ONE giant table with everything:

| order_id | cust_name | cust_email | cust_phone | product | brand |

Customer 'Priya' has 50 orders → her name, email, phone repeated 50 times. She changes her email? Update 50 rows.

Miss one? Inconsistent data.

With normalization: customer data stored ONCE in customers table. Orders table stores just cust_id as a pointer.

Change email = update 1 row. JOINS reconnect the data when you need it.

The Aadhaar-PAN Link

The government says: link your Aadhaar with PAN card. They have two databases. INNER JOIN finds people who exist in BOTH databases. People with only Aadhaar (no PAN) or only PAN (no Aadhaar) are excluded. Only the overlap survives.

INNER JOIN

Technical:

INNER JOIN returns only rows where the JOIN condition is satisfied in BOTH tables. Rows from either table that have no match in the other are excluded. The result is the INTERSECTION based on the join key.

Layman's:

INNER JOIN = 'Show me only people on BOTH guest lists.' If someone is on List A but not List B, they are out. Think Venn diagram — INNER JOIN returns only the overlapping part. JOIN and INNER JOIN are identical.

SYNTAX: INNER JOIN

```
-- Basic INNER JOIN
SELECT a.col1, b.col2
FROM table_a a
INNER JOIN table_b b ON a.key = b.key;

-- JOIN = shorthand for INNER JOIN
SELECT a.col1, b.col2
FROM table_a a
JOIN table_b b ON a.key = b.key;

-- With additional filters
SELECT a.col1, b.col2
FROM table_a a
JOIN table_b b ON a.key = b.key
WHERE a.condition = 'value'
ORDER BY a.col1;

-- IMPORTANT: Always prefix columns with alias
-- to avoid 'ambiguous column' errors!
```

Step-by-Step: How INNER JOIN Works

```
-- orders table:      customers table:
-- order_id | cust_id  customer_id | name
-- -----|-----
-- 1001      | 5        5          | Priya
-- 1002      | 8        8          | Rahul
-- 1003      | 99       12         | Amit

-- INNER JOIN on cust_id = customer_id:
-- order 1001 + cust 5 (Priya)  ✓ MATCH
-- order 1002 + cust 8 (Rahul)  ✓ MATCH
-- order 1003 + cust 99         ✗ NO MATCH
-- customer 12 (Amit)          ✗ NO MATCH

-- Result: 2 rows (only matches survive)
```

PRO TIP

PRO TIP

Table alias convention: use the first letter — o (orders), c (customers), p (products), s (stores). For same-letter tables like stores/shipments, use two letters: st, sh. Always prefix EVERY column with its alias — even unambiguous ones. A future JOIN might add a table with the same column name.

Easy: Orders with Customer Names

EASY

SCENARIO: The dashboard shows order IDs but no customer names. We need to combine sales.orders (has cust_id) with customers.customers (has first_name, last_name) to show who placed each order.

TASK: INNER JOIN orders with customers on cust_id = customer_id. Show order_id, customer name, order_date, status, and net_total. Limit to 10 most recent.

HINT: A JOIN needs an ON condition matching the foreign key in one table to the primary key in the other. Trap: orders uses cust_id but customers uses customer_id — joining on identical names will silently fail because no such column exists in orders.

Answer

✓ ANSWER

```
SELECT
  o.order_id,
  c.first_name || ' ' || c.last_name
    AS customer_name,
  o.order_date,
  o.order_status,
  o.net_total
FROM sales.orders o
INNER JOIN customers.customers c
  ON o.cust_id = c.customer_id
ORDER BY o.order_date DESC
LIMIT 10;
```

Medium: Employees with Department and Store Names

MEDIUM

SCENARIO: The employee table shows dept_id and store_id as numbers. HR wants readable names instead. We need THREE tables: employees, dim_department, and stores.

TASK: Chain two INNER JOINS — employees to dim_department (on dept_id) and employees to stores (on store_id). Show employee name, role, department name, store name, and salary.

HINT: Multi-table JOINS chain — each JOIN adds another bridge from one table you've already joined. Trap: if both tables you're joining have a column with the same name (like store_id), you must qualify it with an alias in SELECT or you'll get an 'ambiguous column' error.

Answer



ANSWER

```
SELECT
  e.first_name || ' ' || e.last_name
    AS employee,
  e.role,
  d.dept_name AS department,
  s.store_name,
  e.salary
FROM stores.employees e
INNER JOIN core.dim_department d
  ON e.dept_id = d.dept_id
INNER JOIN stores.stores s
  ON e.store_id = s.store_id
ORDER BY d.dept_name, e.salary DESC
LIMIT 20;
```

Hard: Delivered Order Details — 4-Table JOIN

HARD

SCENARIO: Build a delivered-order report showing customer name, product name, quantity, and price. This requires FOUR tables: orders → customers, orders → order_items → products.

TASK: Chain three JOINS from orders. Add WHERE for delivered status. Show order_id, date, customer, product, quantity, unit_price, net_amount.

HINT: Map the foreign-key path on paper first — orders connects to customers AND items, items connects to products.

Trap: a one-to-many JOIN (orders → items) multiplies output rows — if you use SUM(net_total) on this expanded result, you'll count the same order's total once per item.

Answer

✓ ANSWER

```
SELECT
  o.order_id,
  o.order_date,
  c.first_name || ' ' || c.last_name
    AS customer,
  p.product_name,
  oi.quantity,
  oi.unit_price,
  oi.net_amount
FROM sales.orders o
INNER JOIN customers.customers c
  ON o.cust_id = c.customer_id
INNER JOIN sales.order_items oi
  ON o.order_id = oi.order_id
INNER JOIN products.products p
  ON oi.prod_id = p.product_id
WHERE o.order_status = 'Delivered'
ORDER BY o.order_date DESC
LIMIT 15;
```

Crazy: Revenue Per Category — JOIN + GROUP BY

CRAZY

SCENARIO: The merchandising team wants revenue per product category. Category lives in dim_category (via brand), revenue lives in order_items. We must JOIN order_items → products → dim_brand → dim_category, then GROUP BY.

TASK: Chain three JOINS from order_items to products, then to dim_brand, then to dim_category. GROUP BY category_name. Show category, items sold, total revenue, and distinct product count. HAVING at least 1000 items. Sort by revenue.

HINT: Category info isn't directly on products — it lives two FK hops away (through dim_brand). Trap: products.products has NO 'category' column; if you reach for p.category your query will fail with 'column does not exist'.

Answer



ANSWER

```
SELECT
    cat.category_name,
    COUNT(*) AS items_sold,
    COUNT(DISTINCT p.product_id)
        AS distinct_products,
    ROUND(SUM(oi.net_amount), 0)
        AS total_revenue,
    ROUND(AVG(oi.unit_price), 0)
        AS avg_unit_price
FROM sales.order_items oi
JOIN products.products p
    ON oi.prod_id = p.product_id
JOIN core.dim_brand b
    ON p.brand_id = b.brand_id
JOIN core.dim_category cat
    ON b.category_id = cat.category_id
GROUP BY cat.category_name
HAVING COUNT(*) >= 1000
ORDER BY total_revenue DESC;
```

INNER JOIN

INNER JOIN returns ONLY matching rows from BOTH tables. The ON clause specifies the match condition (usually PK = FK). Use table aliases for readability. Always prefix columns with aliases. Chain multiple JOINS for 3+ tables. JOIN and INNER JOIN are identical.

INNER JOIN in Every App You Use

Every app you use runs INNER JOINS hundreds of times per second:

- Zomato: Restaurant menu = JOIN restaurants + menu_items + categories
- Amazon: Product page = JOIN products + brands + reviews + inventory
- LinkedIn: Profile = JOIN users + experience + education + skills
- Your bank app: Statement = JOIN accounts + transactions + branches

INNER JOIN is the SQL equivalent of VLOOKUP in Excel — but faster, more flexible, and can chain unlimited tables.

Forgetting ON = Cartesian Product

The Mistake:

`SELECT * FROM orders JOIN customers;` — Without ON, PostgreSQL pairs EVERY order with EVERY customer: $150,000 \times 50,000 = 7.5$ BILLION rows. This is called a Cartesian product and will crash your query.

The Fix:

ALWAYS include ON with the matching key. Also: always prefix columns with aliases to avoid 'ambiguous column' errors when both tables have the same column name (like `store_id`).

The Class Photo

The teacher takes a class photo with EVERY student — even those who forgot their uniform. Some students wear the uniform (matched), some do not (unmatched — NULL for uniform data). LEFT JOIN is this photo: ALL students appear, with uniform data IF it exists, NULL if it does not.

LEFT JOIN

Technical:

LEFT JOIN returns ALL rows from the left table and matching rows from the right. If a left row has no match, right-side columns are NULL. The result always contains AT LEAST as many rows as the left table. LEFT JOIN and LEFT OUTER JOIN are identical.

Layman's:

LEFT JOIN = 'Show me EVERYONE from the left table. If they have a match on the right, include it. If not, still show them with NULL.' Like an attendance register — every name appears, with marks IF they took the exam, or NULL if absent.

SYNTAX: LEFT JOIN

-- Basic LEFT JOIN

```
SELECT a.col1, b.col2
FROM table_a a
LEFT JOIN table_b b ON a.key = b.key;
```

-- Anti-join: find rows with NO match

```
SELECT a.*
FROM table_a a
LEFT JOIN table_b b ON a.key = b.key
WHERE b.key IS NULL;
```

-- LEFT JOIN with aggregates

```
SELECT a.col, COUNT(b.id) AS match_count
FROM table_a a
LEFT JOIN table_b b ON a.key = b.key
GROUP BY a.col;
```

-- CRITICAL: Use COUNT(b.col) not COUNT(*)

-- COUNT(*) returns 1 for unmatched rows!

-- COUNT(b.col) returns 0 (NULL is skipped)!

Step-by-Step: How LEFT JOIN Works

```
-- customers (LEFT):    orders (RIGHT):  
-- customer_id | name  order_id | cust_id  
-- -----|-----  
-- 5           | Priya 1001    | 5  
-- 8           | Rahul 1002    | 8  
-- 12          | Amit  (no order for 12)
```

```
-- LEFT JOIN on customer_id = cust_id:
```

```
-- Priya  + order 1001      ✓ MATCH
```

```
-- Rahul  + order 1002      ✓ MATCH
```

```
-- Amit   + NULL, NULL      ← KEPT!
```

```
-- Result: 3 rows (ALL left rows survive)
```

```
-- Amit's order columns are NULL
```

Easy: All Customers with Order Count

EASY

SCENARIO: Show ALL 50,000 customers with their order count. Include customers who have NEVER placed an order — their count should be 0, not missing. INNER JOIN would skip them entirely.

TASK: LEFT JOIN customers to orders. GROUP BY customer. Use COUNT(o.order_id) not COUNT(*) to get correct counts for zero-order customers.

HINT: Counting in a LEFT JOIN requires care — COUNT(*) counts the LEFT row even when the right side is NULL. Trap: customers with zero orders will look like they have 1 order if you use COUNT(*) — switch to counting the RIGHT-side key to get genuine zeros.

Answer

✓ ANSWER

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
  AS customer,
  COUNT(o.order_id) AS order_count
FROM customers.customers c
LEFT JOIN sales.orders o
  ON c.customer_id = o.cust_id
GROUP BY c.customer_id,
  c.first_name, c.last_name
ORDER BY order_count ASC
LIMIT 15;
```

Medium: Anti-Join — Customers Who Never Ordered

MEDIUM

SCENARIO: Marketing wants a list of customers who registered but NEVER placed an order — for a re-engagement email campaign. This is the 'anti-join' pattern.

TASK: LEFT JOIN customers to orders. Then WHERE o.order_id IS NULL keeps ONLY unmatched customers. Show name, email, registration_date, and years since registration.

HINT: The anti-join pattern — LEFT JOIN keeps everyone, then filter to rows where the right side stayed NULL. Trap: WHERE right_table.any_column = something silently converts your LEFT JOIN back to INNER JOIN — the NULL filter MUST use IS NULL on a right-side column.

Answer



```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  c.email,
  c.registration_date,
  EXTRACT(YEAR FROM
    AGE(c.registration_date))
    AS years_since_reg
FROM customers.customers c
LEFT JOIN sales.orders o
  ON c.customer_id = o.cust_id
WHERE o.order_id IS NULL
ORDER BY c.registration_date;
```

Hard: Product Performance with Sales Tiers

HARD

SCENARIO: Show ALL products with total sales quantity and revenue. Include products that have NEVER been sold (show 0). Classify each product into a sales tier (Best Seller / Popular / Low / Never Sold).

TASK: LEFT JOIN products to order_items on product_id. Use COALESCE to convert NULLs to 0. Use CASE on SUM for tier classification. GROUP BY product.

HINT: When the LEFT side has no matches, aggregates like SUM return NULL — not 0. Trap: a 'Never Sold' tier check must look for NULL FIRST in the CASE; comparing NULL to a number like 'WHEN total = 0' returns UNKNOWN, never TRUE.

Answer (1/2)

✓ ANSWER

```
SELECT
  p.product_id,
  p.product_name,
  p.price AS listed_price,
  COALESCE(SUM(oi.quantity), 0)
    AS total_qty_sold,
  COALESCE(ROUND(
    SUM(oi.net_amount), 0), 0)
    AS total_revenue,
  CASE
    WHEN SUM(oi.quantity) IS NULL
      THEN 'Never Sold'
    WHEN SUM(oi.quantity) < 10
      THEN 'Low Seller'
    WHEN SUM(oi.quantity) < 100
      THEN 'Popular'
    ELSE 'Best Seller'
  END AS sales_tier
FROM products.products p
LEFT JOIN sales.order_items oi
  ON p.product_id = oi.prod_id
GROUP BY p.product_id,
```

Answer (2/2)

 ANSWER

```
p.product_name, p.price  
ORDER BY total_qty_sold DESC  
LIMIT 20;
```

Crazy: Loyalty Program Snapshot — Members, Tiers, Redemptions

CRAZY

SCENARIO: The Loyalty Director wants a snapshot of ALL customers showing: name, current tier, points balance, and total points redeemed lifetime. Include customers who are NOT loyalty members (LEFT JOIN), plus members with zero redemptions. Three tables: customers, loyalty.members, loyalty.tiers, loyalty.redemptions.

TASK: LEFT JOIN customers to loyalty.members, then loyalty.tiers, then aggregate loyalty.redemptions per customer. Show 20 customers ordered by points_balance DESC NULLS LAST.

HINT: Three LEFT JOINs in a row, plus a pre-aggregated subquery for redemptions. Trap: customers who aren't loyalty members will have NULL for tier_name and balance — wrap with COALESCE('Not a Member') for the tier and 0 for the balance, otherwise the report has gaps.

Answer (1/2)

✓ ANSWER

```
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  COALESCE(t.tier_name,
    'Not a Member') AS tier,
  COALESCE(m.points_balance, 0)
    AS points_balance,
  COALESCE(red.total_redeemed, 0)
    AS lifetime_redeemed,
  COALESCE(red.redemption_count, 0)
    AS times_redeemed
FROM customers.customers c
LEFT JOIN loyalty.members m
  ON c.customer_id = m.customer_id
LEFT JOIN loyalty.tiers t
  ON m.tier_id = t.tier_id
LEFT JOIN (
  SELECT customer_id,
    SUM(points_redeemed) AS total_redeemed,
    COUNT(*) AS redemption_count
  FROM loyalty.redemptions
```

Answer (2/2)

✓ ANSWER

```
GROUP BY customer_id  
) red ON c.customer_id = red.customer_id  
ORDER BY m.points_balance DESC  
NULLS LAST LIMIT 20;
```

LEFT JOIN

LEFT JOIN keeps ALL rows from the LEFT table — guaranteed. Unmatched right rows become NULL. Use `COUNT(right_table.col)` not `COUNT(*)` for correct counts. Anti-join: `LEFT JOIN + WHERE right.key IS NULL` = find unmatched rows. `COALESCE(SUM(...), 0)` handles NULLs from unmatched rows.

LEFT JOIN in Production

Anti-join is used everywhere in Indian tech:

- Swiggy: Restaurants with zero orders today → push promotions
- IRCTC: Trains with no bookings → dynamic pricing
- Amazon India: Products with no reviews → review request campaigns
- Flipkart: Sellers who listed products but never got a sale

LEFT JOIN + WHERE IS NULL = your #1 tool for finding MISSING data in production.

WHERE Kills LEFT JOIN Silently

The Mistake:

LEFT JOIN orders o ON ... WHERE o.order_status = 'Delivered' — This removes all NULLs (customers with no orders) because NULL is not equal to 'Delivered'. Your LEFT JOIN silently becomes an INNER JOIN.

The Fix:

Put right-table filters in the ON clause, not WHERE: LEFT JOIN orders o ON c.customer_id = o.cust_id AND o.order_status = 'Delivered'. The filter only applies to the JOIN matching — unmatched left rows are preserved.

The Movie Ticket Counter

The ticket counter has a list of ALL movies playing today (right table). Some movies have advance bookings (left table), some do not. RIGHT JOIN shows ALL movies with booking counts. RIGHT JOIN is the mirror of LEFT JOIN. In practice, most developers use LEFT JOIN and swap the table order instead.

RIGHT JOIN and Multi-Table Patterns

Technical:

RIGHT JOIN returns ALL rows from the right table and matching rows from the left. It is the exact mirror of LEFT JOIN: $A \text{ LEFT JOIN } B = B \text{ RIGHT JOIN } A$. They produce identical results. Multi-table JOINS chain multiple JOIN clauses — each adds columns from a new table.

Layman's:

RIGHT JOIN = LEFT JOIN with tables swapped. Most teams prefer LEFT JOIN exclusively for consistency. Multi-table JOINS follow the FK chain: $\text{orders} \rightarrow \text{customers}$, $\text{orders} \rightarrow \text{order_items} \rightarrow \text{products}$ — each JOIN adds one more table to your query.

SYNTAX: RIGHT JOIN and Multi-Table

```
-- RIGHT JOIN (rarely used in practice)
```

```
SELECT a.col1, b.col2  
FROM table_a a  
RIGHT JOIN table_b b ON a.key = b.key;
```

```
-- Equivalent LEFT JOIN (preferred)
```

```
SELECT a.col1, b.col2  
FROM table_b b  
LEFT JOIN table_a a ON a.key = b.key;
```

```
-- Multi-table JOIN (chain as many as needed)
```

```
SELECT a.col, b.col, c.col, d.col  
FROM table_a a  
JOIN table_b b ON a.x = b.x  
JOIN table_c c ON b.y = c.y  
LEFT JOIN table_d d ON c.z = d.z;
```

```
-- You can mix JOIN types in the same query!
```

LEFT vs RIGHT — Same Result, Different Order

```
-- These two queries return IDENTICAL results:
```

```
-- Using LEFT JOIN (preferred):
```

```
SELECT d.dept_name, COUNT(e.employee_id)
FROM core.dim_department d
LEFT JOIN stores.employees e
      ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

```
-- Using RIGHT JOIN (same result):
```

```
SELECT d.dept_name, COUNT(e.employee_id)
FROM stores.employees e
RIGHT JOIN core.dim_department d
      ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

```
-- Convention: swap table order + use LEFT JOIN
```

Easy: RIGHT JOIN — All Departments with Employee Count

EASY

SCENARIO: Show ALL departments (from `dim_department`) with their employee count. Some departments may have zero employees. RIGHT JOIN keeps all departments even if no employee references them.

TASK: Use `employees` RIGHT JOIN `dim_department`. Count employees per department. Departments with no employees show 0.

HINT: RIGHT JOIN preserves all rows from the right-side table — the 'reverse' of LEFT JOIN. Trap: most teams ban RIGHT JOIN in code reviews because reading 'preserve right' while reading top-to-bottom feels backwards — swap the table order and use LEFT JOIN instead.

Answer

✓ ANSWER

```
SELECT
    d.dept_name,
    COUNT(e.employee_id) AS emp_count
FROM stores.employees e
RIGHT JOIN core.dim_department d
    ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY emp_count DESC;
```

Medium: Store Performance — 2-Table JOIN + Aggregates

MEDIUM

SCENARIO: Per store, show store name, city, total orders, and total revenue. Use LEFT JOIN so stores with zero orders still appear. Only show stores with 50+ orders.

TASK: LEFT JOIN stores with orders on store_id. GROUP BY store. Show store_name, city, order count, total revenue. HAVING at least 50 orders.

HINT: Stores with no orders must survive — that decides your JOIN type. Once aggregated, you'll filter by the resulting count. Trap: putting 'order_count >= 50' in WHERE doesn't work — at WHERE-time the count doesn't exist yet.

Answer

 ANSWER

```
SELECT
    s.store_name,
    s.city,
    COUNT(o.order_id) AS order_count,
    COALESCE(ROUND(
        SUM(o.net_total), 0), 0)
        AS total_revenue
FROM stores.stores s
LEFT JOIN sales.orders o
    ON s.store_id = o.store_id
GROUP BY s.store_id,
    s.store_name, s.city
HAVING COUNT(o.order_id) >= 50
ORDER BY total_revenue DESC
LIMIT 15;
```

Hard: 5-Table Order Detail Report

HARD

SCENARIO: Build a complete order detail: order_id, customer name, store name, product name, quantity, amount, and payment mode. This needs FIVE tables: orders, customers, stores, order_items + products, and payments.

TASK: Chain five JOINS from orders as the anchor. Show the 10 most recent delivered orders with full details.

HINT: Pick the JOIN type per relationship — required dimensions (customer, store, items) use INNER; optional ones (payments may be pending) use LEFT. Trap: a single INNER JOIN to an incomplete table silently drops rows from the entire report — be deliberate about which JOINS filter.

Answer (1/2)

✓ ANSWER

```
SELECT
  o.order_id,
  c.first_name || ' ' || c.last_name
    AS customer,
  s.store_name,
  p.product_name,
  oi.quantity,
  oi.net_amount,
  pay.payment_mode
FROM sales.orders o
JOIN customers.customers c
  ON o.cust_id = c.customer_id
JOIN stores.stores s
  ON o.store_id = s.store_id
JOIN sales.order_items oi
  ON o.order_id = oi.order_id
JOIN products.products p
  ON oi.prod_id = p.product_id
LEFT JOIN sales.payments pay
  ON o.order_id = pay.order_id
WHERE o.order_status = 'Delivered'
ORDER BY o.order_date DESC
```

Answer (2/2)

 ANSWER

```
LIMIT 10;
```

Crazy: Call Center Performance — Agent + Customer + Tickets

CRAZY

SCENARIO: The Customer Care Head wants a multi-table report from the call center system. Per agent, show: agent name, store, total calls handled, avg call duration, total tickets resolved, and customer count contacted. Four tables: call_center.calls, support.tickets, stores.employees, stores.stores.

TASK: Start from employees. LEFT JOIN pre-aggregated call_center.calls per agent (count, avg duration, distinct customers). LEFT JOIN pre-aggregated support.tickets per agent (resolved count). JOIN stores for branch name. Show top 15 by call volume.

HINT: Two separate one-to-many tables (calls AND tickets) joined to the same employee — pre-aggregate each in its own subquery to avoid fan-out. Trap: an agent with zero calls but several tickets would lose their ticket data with INNER JOIN — LEFT JOIN preserves all employees.

Answer (1/2)

✓ ANSWER

```
SELECT
  e.first_name || ' ' || e.last_name
    AS agent,
  s.store_name,
  COALESCE(c.call_count, 0) AS calls,
  ROUND(COALESCE(c.avg_duration, 0), 0)
    AS avg_call_secs,
  COALESCE(c.unique_customers, 0)
    AS unique_callers,
  COALESCE(t.tickets_resolved, 0)
    AS tickets_resolved
FROM stores.employees e
JOIN stores.stores s
  ON e.store_id = s.store_id
LEFT JOIN (
  SELECT agent_id,
    COUNT(*) AS call_count,
    AVG(call_duration_seconds)
      AS avg_duration,
    COUNT(DISTINCT customer_id)
      AS unique_customers
  FROM call_center.calls
```

Answer (2/2)

✓ ANSWER

```
GROUP BY agent_id
) c ON e.employee_id = c.agent_id
LEFT JOIN (
  SELECT agent_id,
    COUNT(*) FILTER (
      WHERE status = 'Resolved')
    AS tickets_resolved
  FROM support.tickets
  GROUP BY agent_id
) t ON e.employee_id = t.agent_id
ORDER BY calls DESC LIMIT 15;
```

RIGHT JOIN and Multi-Table JOINS

RIGHT JOIN = LEFT JOIN with tables swapped. Prefer LEFT JOIN for consistency. Multi-table JOINS chain JOIN clauses — each adds columns from a new table. Use COUNT(DISTINCT) or subqueries when multiple JOINS expand rows. Mix JOIN types in the same query (INNER for required links, LEFT for optional).

PRO TIP

If your JOIN returns more rows than expected, a one-to-many relationship is expanding results. Fix: use COUNT(DISTINCT) instead of COUNT(*), or pre-aggregate in a subquery. If it returns fewer rows than expected, an INNER JOIN is dropping unmatched rows — switch to LEFT JOIN.

Try It Live!

Run each query on your RetailMart database. Change the JOIN type. Swap LEFT for INNER and see what disappears. Understanding comes from experimentation.

Challenge 1: Products with Brand Names

```
-- Show products with their brand name
SELECT p.product_id, p.product_name,
       p.price, b.brand_name
FROM products.products p
JOIN core.dim_brand b
      ON p.brand_id = b.brand_id
ORDER BY p.price DESC LIMIT 15;

-- YOUR TURN: Add dim_category to show
-- category_name too (chain another JOIN)
```

Challenge 2: Orders with Store City

```
-- Show orders with the store city
SELECT o.order_id, o.net_total,
       s.store_name, s.city
FROM sales.orders o
JOIN stores.stores s
      ON o.store_id = s.store_id
ORDER BY o.net_total DESC LIMIT 15;

-- YOUR TURN: Add a LEFT JOIN to customers
-- to show customer name too
```

Challenge 3: Unsold Products (Anti-Join)

```
-- Find products that were NEVER sold
SELECT p.product_id, p.product_name,
       p.price
FROM products.products p
LEFT JOIN sales.order_items oi
      ON p.product_id = oi.prod_id
WHERE oi.order_item_id IS NULL
ORDER BY p.price DESC;

-- YOUR TURN: Count how many products
-- have never been sold
```

Challenge 4: Store Revenue Ranking

```
-- Show each store with its total revenue
SELECT s.store_name, s.city,
       ROUND(SUM(o.net_total), 0)
       AS revenue
FROM stores.stores s
JOIN sales.orders o
      ON s.store_id = o.store_id
GROUP BY s.store_id,
         s.store_name, s.city
ORDER BY revenue DESC LIMIT 10;

-- YOUR TURN: Change to LEFT JOIN
-- to include stores with zero orders
```

Challenge 5: Customer Orders + Payment Mode

```
-- Show customer orders with payment info
SELECT c.first_name, o.order_id,
       o.net_total, pay.payment_mode
FROM sales.orders o
JOIN customers.customers c
      ON o.cust_id = c.customer_id
LEFT JOIN sales.payments pay
      ON o.order_id = pay.order_id
ORDER BY o.net_total DESC LIMIT 15;

-- YOUR TURN: Add the store name too
-- (chain another JOIN to stores)
```

PRO TIP

JOIN cheat sheet: (1) Need ONLY matches? → INNER JOIN. (2) Need ALL from one side? → LEFT JOIN. (3) Need to find what is MISSING? → LEFT JOIN + WHERE IS NULL. (4) Need 3+ tables? → Chain JOINS following FK → PK paths. When in doubt, start with LEFT JOIN — it never hides data.

Ambiguous Column Error

The Mistake:

`SELECT store_id, ... FROM orders JOIN stores` — ERROR! Both tables have `store_id`. PostgreSQL does not know which one you mean.

The Fix:

Always prefix: `SELECT o.store_id or s.store_id`. Use short aliases (o, s, c, p) and prefix EVERY column. This prevents ambiguity and makes queries readable.

Joining on Wrong Column

The Mistake:

JOIN customers ON o.customer_id = c.customer_id — sales.orders uses cust_id, not customer_id. The query returns zero rows or crashes.

The Fix:

Check actual column names: ON o.cust_id = c.customer_id. RetailMart uses cust_id in orders but customer_id in customers. Always verify FK column names.

Row Multiplication in Multi-Table JOINS

The Mistake:

JOIN employees AND orders to the same stores table. If a store has 10 employees and 500 orders, you get 5,000 rows per store. SUM(net_total) is now 10× inflated.

The Fix:

Pre-aggregate in subqueries: LEFT JOIN (SELECT store_id, COUNT(*) FROM employees GROUP BY store_id) emp ON ...

This keeps one row per store and correct totals.

Using **SELECT *** in JOINS

The Mistake:

`SELECT * FROM orders JOIN customers ON ...` — Returns ALL columns from BOTH tables, including duplicate keys (`cust_id` AND `customer_id`), internal timestamps, and columns you do not need. Slow and confusing.

The Fix:

Always list specific columns with aliases: `SELECT o.order_id, c.first_name, o.net_total`. This is faster (less data), clearer (you know what you get), and avoids ambiguous column errors.

Joining Tables Without Understanding the Relationship

The Mistake:

Blindly joining tables that are not directly related. Example: JOIN customers directly with products — there is no FK between them. The correct path is: customers → orders → order_items → products.

The Fix:

Always trace the FK chain first. Draw it out: orders.cust_id → customers.customer_id, order_items.prod_id → products.product_id. If there is no FK path, the JOIN probably does not make sense.

INNER JOIN vs LEFT JOIN

Q: 'What is the difference between INNER JOIN and LEFT JOIN?'

A: INNER JOIN returns only rows that match in BOTH tables. LEFT JOIN returns ALL rows from the left table, with NULLs for unmatched right-side columns. If every left row has a match, both produce the same result. They differ only when unmatched rows exist.

Anti-Join Pattern

Q: 'How do you find rows in Table A with no match in Table B?'

A: Anti-join pattern: `SELECT a.* FROM table_a a LEFT JOIN table_b b ON a.key = b.key WHERE b.key IS NULL`. This finds unmatched rows efficiently. Use cases: customers who never ordered, products never reviewed, employees never assigned tickets.

Cartesian Product

Q: 'What happens if you JOIN without an ON clause?'

A: You get a Cartesian product — every row from Table A paired with every row from Table B. For 150K orders and 50K customers, that is 7.5 BILLION rows. Always include ON. This is the #1 most dangerous JOIN mistake.

ON vs WHERE in LEFT JOIN

Q: 'Where should you put filters in a LEFT JOIN?'

A: In LEFT JOIN, putting a right-table filter in WHERE drops unmatched rows (silently converts to INNER JOIN). Putting it in ON preserves unmatched rows. Example: LEFT JOIN orders ON ... AND status='Delivered' keeps all customers. WHERE status='Delivered' drops zero-order customers.

JOIN and NULL Handling

Q: 'How do you handle NULLs from LEFT JOIN in aggregates?'

A: Use `COUNT(right_table.col)` instead of `COUNT(*)` — it returns 0 for unmatched rows. Use `COALESCE(SUM(...), 0)` to convert NULL totals to 0. Use `COALESCE(AVG(...), 0)` for averages. NULLs from unmatched rows are the #1 source of wrong aggregates.

Row Explosion in Multi-Table JOINS

Q: 'Why does my JOIN return more rows than expected?'

A: One-to-many relationships expand rows. If one order has 5 items, the JOIN produces 5 rows per order. Fix: use COUNT(DISTINCT) or pre-aggregate in a subquery. Check: compare COUNT(*) vs COUNT(DISTINCT order_id) — if they differ, rows are multiplied.

NATURAL JOIN — Why to Avoid It

Q: 'What is NATURAL JOIN and should you use it?'

A: NATURAL JOIN automatically matches columns with the same name in both tables. Do NOT use it in production — it is fragile. If someone adds a column with a matching name, your query silently changes behavior. Always use explicit ON clauses for safety and clarity.

Performance: INNER vs LEFT JOIN

Q: 'Is INNER JOIN faster than LEFT JOIN?'

A: Often yes. INNER JOIN lets the optimizer eliminate unmatched rows early. LEFT JOIN must preserve all left rows, reducing optimization options. But the difference is usually small. Choose based on correctness first — do you NEED unmatched rows? If yes, LEFT JOIN. If not, INNER JOIN.

Real Interview: Customer Segmentation

Q: 'Write a query to segment customers by order activity: VIP (10+ orders, 50K+ spend), Regular (3-9 orders), New (1-2 orders), Inactive (0 orders).'

A: LEFT JOIN customers to orders, GROUP BY customer, COUNT(o.order_id) for orders, SUM(o.net_total) for spend, CASE for tier. LEFT JOIN includes inactive (zero-order) customers. This combines earlier sessions in one query.

KEY TAKEAWAYS

INNER JOIN:

1. Returns ONLY rows matching in BOTH tables — the intersection.
2. JOIN = INNER JOIN (shorthand). Always include ON clause.

LEFT JOIN:

3. Returns ALL from left + matches from right. Unmatched → NULL.
4. Anti-join: LEFT JOIN + WHERE right.key IS NULL = find missing data.
5. Use COUNT(right.col) not COUNT(*) for correct zero counts.
6. Right-table filters go in ON, not WHERE (or LEFT becomes INNER).

MULTI-TABLE:

7. Chain JOINS by following the FK → PK path.
8. Mix JOIN types: INNER for required, LEFT for optional.
9. Watch for row multiplication — use DISTINCT or subqueries.

What You Achieved Today

You mastered the THREE core JOINS: INNER (only matches), LEFT (all from left), RIGHT (mirror of left). You learned multi-table JOINS connecting 3, 4, 5+ tables. You know the anti-join pattern to find missing data.

Every real query uses JOINS. Amazon's order page, Swiggy's menu, Zomato's reviews — all powered by JOINS across many tables. This is the skill that separates SQL learners from SQL professionals.

Tomorrow: FULL OUTER JOIN, CROSS JOIN, SELF JOIN, and advanced JOIN patterns. This completes your JOIN toolkit!

SQL JOINS Part 1

-- INNER JOIN

```
FROM a JOIN b ON a.key = b.key
-- Only matching rows from BOTH tables
-- JOIN = shorthand for INNER JOIN
```

-- LEFT JOIN

```
FROM a LEFT JOIN b ON a.key = b.key
-- ALL from left + matches from right
-- NULLs for unmatched right rows
```

-- RIGHT JOIN

```
FROM a RIGHT JOIN b ON a.key = b.key
-- Same as: FROM b LEFT JOIN a ...
-- Rarely used; prefer LEFT JOIN
```

SQL JOINS Part 1

-- Anti-Join Pattern

```
FROM a LEFT JOIN b ON a.key = b.key
WHERE b.key IS NULL
-- Find rows in A with NO match in B
```

-- Multi-Table JOIN

```
FROM a JOIN b ON a.x = b.x
      JOIN c ON b.y = c.y
-- Chain JOINS: follow FK to PK
```

-- LEFT JOIN Filter Rule

```
ON a.key = b.key AND b.col = 'X'
-- Preserves unmatched left rows
WHERE b.col = 'X' -- KILLS LEFT JOIN!
```

Joins Part 1 Take-Home Challenge (1/3)

Build these queries on RetailMart V3:

Easy (Two-table JOINS):

- 1.: JOIN orders with customers. Show customer name, order_date, net_total. LIMIT 20.
- 2.: JOIN employees with stores. Show employee name, role, store_name, city.
- 3.: JOIN products with dim_brand. Show product_name, price, brand_name.

Joins Part 1 Take-Home Challenge (2/3)

Medium (JOIN + Aggregates):

- 4.:** Per customer: name, total orders, total spend. LEFT JOIN for zero-order customers.
- 5.:** Per department: dept_name, employee count, avg salary. Include all departments.
- 6.:** Per product: product_name, total quantity sold, revenue. LEFT JOIN for unsold products.

Hard (Multi-table + Anti-join):

- 7.:** Find customers who registered but never ordered (anti-join). Show name, email, date.
- 8.:** Order details with 4 tables: order_id, customer, product, quantity, amount. Delivered only.

Crazy:

- 9.:** Customer segmentation: name, orders, spend, avg value, tier (VIP/Regular/New/Inactive). LEFT JOIN + CASE + aggregates.

CHALLENGE

Joins Part 1 Take-Home Challenge (3/3)

Push your SQL file to GitHub!

<https://github.com/starlordali4444/postgresql>

Coming Tomorrow

FULL OUTER JOIN — keep ALL rows from BOTH tables. CROSS JOIN — every combination (like generating a calendar).

SELF JOIN — join a table to itself (employee → manager hierarchy).

Joins Part 2 completes your JOIN toolkit. After that, you can connect ANY tables in ANY combination. Combined with earlier sessions skills, you can build any business query.